

Descargas

DESCARGAR DOCKER [Docker Desktop: The #1 Containerization Tool for Developers | Docker](#)

Tutoriales

- DOCKER PARA WINDOWS [Instalar Docker en Windows 11: paso a paso - IONOS España](#)
- DOCKER PARA LINUX [Tutorial de Docker - atareao con Linux](#)
- TUTORIALES DE COMANDOS DOCKER [brunocascio/docker-espanol: Un tutorial Docker en español. Basado en el libro Docker Cookbook de O'reilly \(github.com\)](#)

Enlaces de Interés

- Docker: Sitio oficial <https://www.docker.com/>
- DockerHub: Imágenes [Explore Docker's Container Image Repository | Docker Hub](#)
- Docker File: Manejo de Dockerfile <https://docs.docker.com/build/concepts/dockerfile/>
- Docker Compose: Manejo de Compose [Docker Compose | Docker Docs](#)
- Curso de Docker: Libro en PDF de José Juan Sánchez Hernández <https://josejuansanchez.org/iaw/practica-docker/index.pdf>

Comandos principales de Dockers

docker run

El comando **docker run** se utiliza para crear y ejecutar [contenedores](#) a partir de imágenes de Docker. A continuación, se muestra la sintaxis básica del comando:

`1 docker run [opciones] <imagen> [comando]`

- [opciones]: Las opciones permiten configurar diferentes aspectos del contenedor, como la asignación de puertos, variables de entorno, volúmenes, etc.
- <imagen>: Especifica la imagen a partir de la cual se creará el contenedor.
- [comando]: Es opcional y permite ejecutar un comando específico dentro del contenedor.

Algunas opciones comunes del comando **docker run** incluyen:

- -d: Ejecuta el contenedor en segundo plano (modo daemon).
- -p <puerto_host>:<puerto_contenedor>: Mapea un puerto del host al puerto del contenedor.
- -e <variable>=<valor>: Define variables de entorno dentro del contenedor.

Veamos un ejemplo de uso del comando **docker run**:

`1 docker run -d -p 8080:80 nginx`

Este comando crea y ejecuta un contenedor a partir de la imagen de **Nginx**, asignando el puerto **8080** del host al puerto **80** del contenedor. El contenedor se ejecuta en segundo plano (modo daemon).

docker build

El comando **docker build** nos permite construir nuestras propias imágenes de Docker a partir de un archivo llamado **Dockerfile**. El **Dockerfile** es un archivo de texto que contiene una serie de instrucciones que describen los pasos necesarios para construir la imagen.

La sintaxis básica del comando **docker build** es la siguiente:

```
1 docker build [opciones] -t <nombre_imagen> <ruta_contexto>
```

- [opciones]: Permite configurar diferentes aspectos del proceso de construcción, como la utilización de una caché, la especificación del **Dockerfile** alternativo, etc.
- -t <nombre_imagen>: Especifica el nombre y la etiqueta de la imagen resultante.
- <ruta_contexto>: Especifica la ruta al contexto de construcción, que incluye el **Dockerfile** y cualquier archivo necesario.

Veamos un ejemplo de uso del comando **docker build**:

```
1 docker build -t mi_aplicacion:1.0
```

Este comando construye una imagen llamada **mi_aplicacion** con la etiqueta **1.0** a partir del **Dockerfile** ubicado en el directorio actual (.).

docker pull y docker push

Los comandos **docker pull** y **docker push** se utilizan para descargar y subir imágenes a un registro de Docker, como Docker Hub.

El comando **docker pull** se utiliza para descargar una imagen desde un registro. La sintaxis básica es la siguiente:

```
1 docker pull <nombre_imagen>
```

- <nombre_imagen>: Especifica el nombre y la etiqueta de la imagen a descargar.

Por ejemplo, para descargar la última versión de la imagen de **Ubuntu**, ejecutamos el siguiente comando:

```
1 docker pull ubuntu
```

El comando **docker push** se utiliza para subir una imagen a un registro. Antes de utilizar este comando, es necesario etiquetar la imagen con el nombre del registro al que se va a subir. La sintaxis básica es la siguiente:

```
1 docker push <nombre_imagen>
```

- <nombre_imagen>: Especifica el nombre y la etiqueta de la imagen a subir.

Supongamos que hemos creado una imagen personalizada llamada **mi_aplicacion** con la etiqueta **1.0**. Para subir esta imagen a Docker Hub, ejecutamos los siguientes comandos:

```
1 docker tag mi_aplicacion:1.0 usuario/dockerhub-repo:1.0
```

```
2docker push usuario/dockerhub-repo:1.0
```

El primer comando etiqueta la imagen con el nombre del registro y el segundo comando la sube al registro especificado.

docker images

El comando **docker images** se utiliza para mostrar una lista de las imágenes disponibles en el sistema. La sintaxis básica es la siguiente:

```
1docker images [opciones]
```

- [opciones]: Permite filtrar y formatear la salida de las imágenes.

Algunas opciones comunes del comando **docker images** incluyen:

- -a: Muestra todas las imágenes, incluyendo las intermedias y las no utilizadas.
- --filter: Permite filtrar las imágenes según diferentes criterios, como la etiqueta, el nombre, etc.

Por ejemplo, para mostrar todas las imágenes disponibles en el sistema, ejecutamos el siguiente comando:

```
1docker images -a
```

Este comando muestra una lista de todas las imágenes, incluyendo las intermedias y las no utilizadas.

docker ps

El comando **docker ps** se utiliza para mostrar una lista de los contenedores en ejecución. Esta es la sintaxis básica del comando:

```
1docker ps[opciones]
```

Algunas opciones comunes del comando **docker ps** incluyen:

- -a: Muestra todos los contenedores, incluyendo los que no están en ejecución.
- -q: Muestra solo los identificadores numéricos de los contenedores.
- -l: Muestra el último contenedor creado.

Por ejemplo, para ver una lista de los contenedores en ejecución, ejecutamos el siguiente comando:

```
1docker ps
```

Este comando mostrará información como el ID del contenedor, la imagen utilizada, el estado, los puertos asignados, entre otros.

docker stop

El comando **docker stop** se utiliza para detener uno o varios contenedores en ejecución. La sintaxis básica es la siguiente:

```
1docker stop [opciones] <nombre_contenedor>...
```

- [opciones]: Permite configurar el tiempo de espera para la finalización del contenedor y otras opciones.
- <nombre_contenedor>: Especifica el nombre o el ID del contenedor a detener.

Por ejemplo, para detener un contenedor con el nombre "mi_contenedor", ejecutamos el siguiente comando:

```
1 docker stop mi_contenedor
```

Si deseamos detener varios contenedores a la vez, simplemente agregamos los nombres o los IDs de los contenedores separados por espacios.

docker rm y docker rmi

El comando **docker rm** se utiliza para eliminar uno o varios contenedores. La sintaxis básica es la siguiente:

```
1 docker rm [opciones] <nombre_contenedor>...
```

- [opciones]: Permite configurar diferentes aspectos del comando, como [eliminar contenedores](#) en ejecución y otras opciones.
- <nombre_contenedor>: Especifica el nombre o el ID del contenedor a eliminar.

Por ejemplo, para eliminar un contenedor con el nombre "mi_contenedor", ejecutamos el siguiente comando:

```
1 docker rm mi_contenedor
```

Si deseamos eliminar varios contenedores a la vez, simplemente agregamos los nombres o los IDs de los contenedores separados por espacios.

El comando **docker rmi** se utiliza para eliminar una o varias imágenes. La sintaxis básica es similar a la del comando **docker rm**:

```
1 docker rmi [opciones] <nombre_imagen>...
```

- [opciones]: Permite configurar diferentes aspectos del comando, como forzar la eliminación de imágenes en uso y otras opciones.
- <nombre_imagen>: Especifica el nombre o la etiqueta de la imagen a eliminar.

Por ejemplo, para eliminar una imagen con el nombre "mi_imagen:1.0", ejecutamos el siguiente comando:

```
1 docker rmi mi_imagen:1.0
```

Si deseamos eliminar varias imágenes a la vez, simplemente agregamos los nombres o las etiquetas de las imágenes separadas por espacios.

docker network

El comando **docker network** se utiliza para gestionar las redes en Docker. Proporciona una forma de crear y configurar redes virtuales para permitir la comunicación entre contenedores.

Algunos comandos útiles del comando **docker network** incluyen:

- `docker network ls`: Muestra una lista de las redes disponibles.
- `docker network create`: Crea una nueva red.
- `docker network connect`: Conecta un contenedor a una red existente.
- `docker network disconnect`: Desconecta un contenedor de una red.

Por ejemplo, para crear una nueva red llamada "mi_red", ejecutamos el siguiente comando:

```
1 docker network create mi_red
```

Luego, podemos conectar un contenedor existente a esta red utilizando el siguiente comando:

```
1 docker network connect mi_red mi_contenedor
```

docker volume

El comando **docker volume** se utiliza para gestionar los volúmenes en Docker. Los volúmenes son una forma de persistir datos y compartirlos entre los contenedores y el host. A continuación, se muestra la sintaxis básica del comando:

```
1 docker volume [opciones] <subcomando>
```

Algunos subcomandos útiles del comando **docker volume** incluyen:

- `create`: Crea un nuevo volumen.
- `ls`: Muestra una lista de los volúmenes disponibles.
- `inspect`: Muestra información detallada sobre un volumen específico.
- `rm`: Elimina uno o varios volúmenes.

Por ejemplo, para crear un nuevo volumen llamado "mi_volumen", ejecutamos el siguiente comando:

```
1 docker volume create mi_volumen
```

Podemos ver una lista de los volúmenes disponibles utilizando el siguiente comando:

```
1 docker volume ls
```

Si deseamos eliminar un volumen, utilizamos el siguiente comando:

```
1 docker volume rm mi_volumen
```

docker-compose

El comando **docker-compose** se utiliza para definir y gestionar aplicaciones multi-contenedor utilizando un archivo de configuración llamado **docker-compose.yml**. Este archivo describe los servicios, las redes y los volúmenes necesarios para la aplicación. A continuación, se muestra la sintaxis básica del comando:

```
1 docker-compose [opciones] <comando>
```

Algunos comandos comunes del comando **docker-compose** incluyen:

- up: Crea y ejecuta los servicios definidos en el archivo **docker-compose.yml**.
- down: Detiene y elimina los servicios definidos en el archivo **docker-compose.yml**.
- restart: Reinicia los servicios definidos en el archivo **docker-compose.yml**.
- ps: Muestra el estado de los servicios definidos en el archivo **docker-compose.yml**.

Para utilizar **docker-compose**, primero debemos crear un archivo **docker-compose.yml** con la configuración adecuada para nuestra aplicación. Luego, podemos ejecutar el siguiente comando para iniciar los servicios:

```
1 docker-compose up
```

Si deseamos detener y eliminar los servicios, utilizamos el siguiente comando:

```
1 docker-compose down
```